



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK



Budapest,
2026.05.19.

SZÁMÍTÓGÉP ARCHITEKTÚRÁK

Támadások a spekulatív végrehajtás ellen

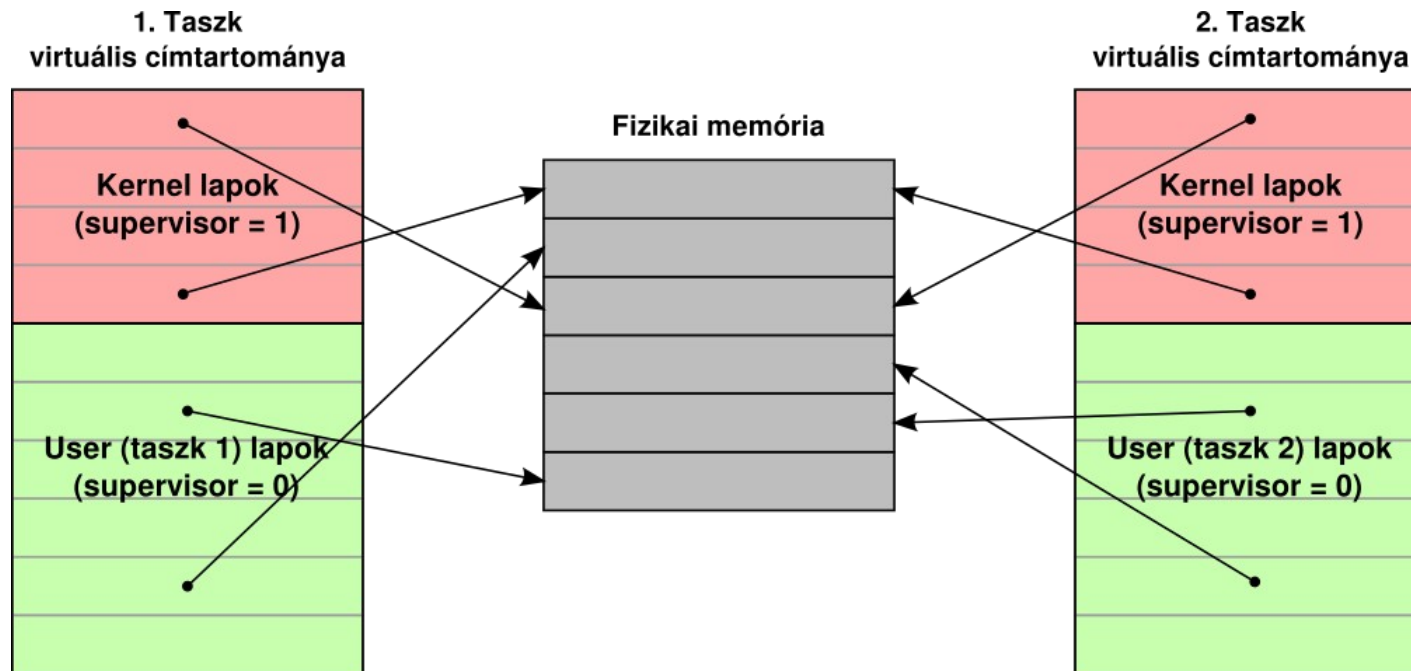
Horváth Gábor, Belső Zoltán

BME Hálózati Rendszerek és Szolgáltatások Tanszék
ghorvath@hit.bme.hu, belso@hit.bme.hu



Hozzávalók

- Ismétlés: minden taszk külön laptábla
 - Taszk váltás → laptábla váltás
- Amiről nem beszéltünk:
 - A taszkok laptáblái tartalmazzák a kernel címteret is
 - Hogy a kernelt gyorsan lehessen hívni
 - ... de ezeken a lapokon a „supervisor” bit=1 → csak a kernel éri el



- User taszk:
 - Csak a saját lapjait éri el
- Kernel:
 - A taszk és kernel területet is eléri
- Kernel hívás:
 - User taszk egy speciális megszakítással hívja a kernelt
 - EAX regiszterbe tesszük a kívánt kernel funkció számát
 - Linux: **INT 0x80**, Windows: **INT 0x2e**
 - Megszakításkor CPU automatikusan user → kernel módba vált
 - Elérhetők lesznek a kernel lapok is
 - Kernel hívás végén visszavált user módba

- Spekulatív végrehajtás =
olyan utasítás hajtódik végre, ami lehet, hogy nem is szükséges
- Két megjelenési formája:
 - Elágazásbecslés
 - Rossz becslés: téves utasítások „invalidálása”
 - Kivételkezelés
 - Kivétel fellép: a tévedésből elkezdett utasítások „invalidálása”
- Invalidálás jelentése:
 - Téves utasítások eredményének eldobása
 - Az architektúrális állapot szintjén
 - ... ennyi nem mindig elég !!! Nyomtalanul el kellene tűnnie!
 - Mikroarchitektúrális állapot nem mindig visszaállítható
 - Nyomot hagyhat cache-ben, TLB-ben, BTB-ben, stb.
 - Szerencsére ezek nem elérhetőek (biztos?)

- = side channel attack
- = próbálgatunk és mérünk
- N karakteres jelszó törése
 26^N helyett $26 \cdot N$ lépésben:
 - Próbálgatunk: Ax, Bx, Cx, ..., Zx
 - Mérjük a reakcióidőt
 - Tovább tart a válasz, ha eltaláltuk az első betűt
 - Strcmp-nek a másodikat is meg kell néznie!
 - Továbblépünk a második betűre, stb.



A complex network diagram with numerous nodes of varying sizes and colors (grey, white, and orange) connected by thin lines. Some nodes are highlighted with dashed circles. The background is a light grey gradient.

Meltdown



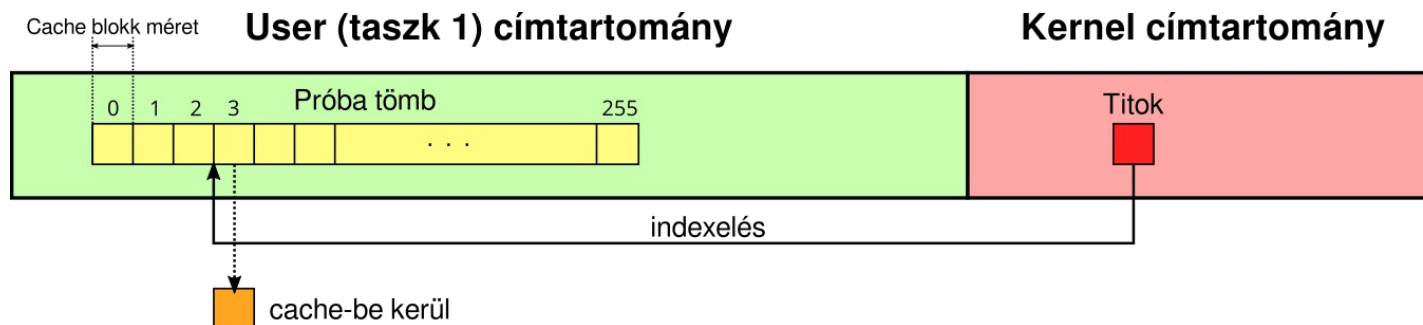
- A probléma gyökere:
 - A támadó kihasználja a spekulatív végrehajtást
 - Spekulatív utasítások mikroarchitekturális nyomot hagyhatnak
 - Ezt kihasználva elérhető a kernel memória
- Ötlet:
 - Próbáljunk meg elérni kernel adatot (titkot)
 - Nem fog menni (védelmi hiba)
 - De az adatelérés lefuthatott, mielőtt a védelmi hiba kiderül
 - Persze invalidálva lesz, de a cache-ben nyoma marad
 - ... ezt használjuk ki!

- Kernel titok, amit el akarunk érni: „titok”

i1: $R1 \leftarrow \text{MEM}[\text{titok címe}]$

i2: $R2 \leftarrow R1 * 64$

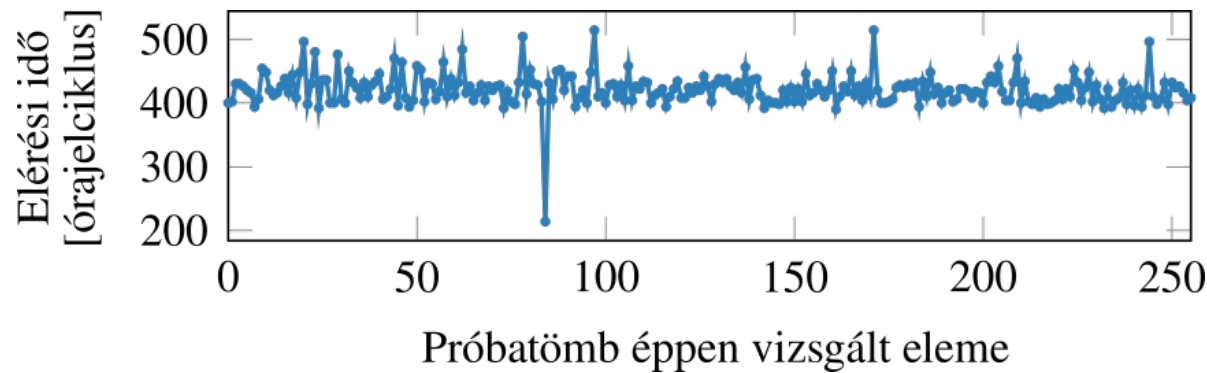
i3: $R3 \leftarrow \text{MEM}[\text{proba} + R2]$



- i1: védelmi hibát okoz, nem hajtódik végre tovább a program
- De a hiba csak később derül ki
- i2, i3 invalidálásra kerül (R1, R2, R3 nem változik)
- De a próba tömb R1-edik eleme a cache-be került!**

(64 helyett jobb 4096-tal szorozni, kisebb lesz a zaj)

- **A próba tömb R1-edik eleme a cache-be került!**
(miközben a tartalma persze nem változott)
- A próba tömb elemei 64 byte-osak
→ minden R1 értéknél más blokk kerül a cache-be
- Kimérhetjük, mi lehetett R1!
- Végigmegyünk a próba tömbön, és mérjük az elérési időt:



→ R1 értéke 84 volt

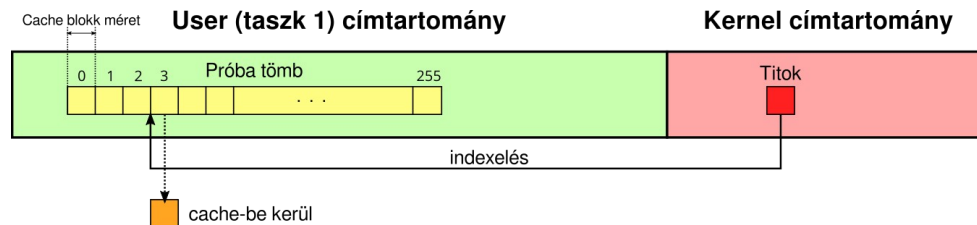
- Kernel memória 100-500 KB/s segítségével dumpolható
- KPTI (KASLR, KAISER)
 - Kernel lapokat nem tesszük a user taszk laptáblájába
 - → Kernelhíváskor is laptáblát kell váltani
 - Akár 30% overhead is lehet
 - AMD CPU-k nem érintettek

- Meltdown (2017):
 - A kivételkezelés működését használja ki
 - A taszk saját laptábláján keresztül ér el adatot jogosulatlanul
 - Könnyen gyógyítható ($\approx$)
- Spectre (2017):
 - Feltételes elágazás miatti spekulatív végrehajtást használja ki
 - Cél: a CPU rábírása, hogy spekulatívan végrehajtsa egy általunk választott kódot egy idegen címtérben
 - Spectre v1: ugrási feltétel becslése érintett
 - Spectre v2: ugrási cím becslése érintett (BTB szándékos „félretanítása”)
 - Idegen taszk laptábláján keresztül jut titokhoz
 - Nagyon nehezen kezelhető
- Sorra derül fény egyéb, spekulatív mechanizmusok támadhatóságára!

- Kell hozzá egy feltételes elágazás:

```
if (x < array1_size) {  
    y = array2[array1[x] * 64];  
}
```

- Jó sokszor meghívjuk, hogy megtanulja: a feltétel mindig teljesül
- ...aztán meghívjuk úgy, hogy **array1[x]** kernel területre essen (a titokra!)
(ilyenkor persze x-ez az array1_size-nál nagyobbra állítjuk – ott vannak a titkok)
 - Elágazásbecslő „nem ugrás”-t jósol
 - **array1[x]** spekulatív kiolvasódik, **array2** megfelelő blokkja a cache-be kerül
 - ...persze utólag nem érvényesül a programsor
 - **De az array2 próba tömb array1[x]-edik eleme a cache-be került!**



- Ki tudjuk mérni elérési idővel, hogy melyik!
(64 helyett jobb 4096-tal szorozni, kisebb lesz a zaj)



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

